Section 23 : Bonus Section - Enumerations

Information

- Title: `Bonus Section - Enumerations`
- Authors: `Etienne KOA`

Bonus Section Overview

`C++` Énumérations

- Qu'est-ce qu'une énumération ?
  - Motivation
- Structure d'une énumération
- Type d'énumérations
  - Énumération sans portée
  - Énumération ciblée
- Énumérations en cours d'utilisation

Motivation

- Qu'est-ce qu'une énumération ?
  - Un type défini par l'utilisateur qui modélise un ensemble de valeurs intégrales constantes
    - Les jours de la semaine (Mon, Tue, Wed, ...)
    - Les mois de l'année (Jan, Feb, Mar, ...)
    - Les couleurs dans un jeu de cartes (Clubs, Hearts, Spades, Diamonds)
    - Les valeurs dans un jeu de cartes (Ace, Two, Three, ...)
    - États du système (Idle, Defense_Mode, Attack_Mode, ...)
    - Les directions sur une boussole (North, South, East, West)
- Avant les types énumérés
  - Constantes numériques sans nom
  - `Nombres magiques`
- Ces constantes seraient utilisées comme conditions dans les instructions de contrôle
- Souvent, on n'aurait aucune idée de ce que faisait un algorithme
- En conséquence, de nombreux algorithmes souffraient d'une faible lisibilité et d'un nombre élevé d'erreurs logiques.

Lisibilité

```
int state;
std::cin >> state;

if (state == 0)
    initiate(3);
else if (state == 1)
    initiate(4);
else if (state == 2)
    initiate(5);
```

```
enum State {Engine_Failure = 0, Inclement_Weather = 1, Nominal = 2};
enum Sequence {Abort = 3, Hold = 4, Launch = 5};

int user_input;
std::cin >> user_input;
State state = State(user_input);

if (state == Engine_Failure)                    // state = 0
    initiate(Abort);                            // sequence = 3
else if (state == Inclement_Weather)            // state = 1
    initiate(Hold);                             // sequence = 4
else if (state == Nominal)                      // state = 2
    initiate(Launch);                           // sequence = 5
```

Exactitude algorithmique

```
int state = get_state();

if (state == 0)
    initiate(3);
else if (state == 1)
    initiate(4);
else if (state == 2)
    initiate(5);
```

```
int get_state(){
    return state_of_fridge;
}

int getState(){
    return state_of_rocket;
}
```

Compiles

```
State state = get_state();

if (state == Engine_Failure)
    initiate(Abort);
else if (state == Inclement_Weather)
    initiate(Hold);
else if (state == Nominal)
    initiate(Launch);
```

```
int get_state(){
    return state_of_fridge;
}

State getState(){
    return state_of_rocket;
}
```

Does not compile

La structure d'une énumération

La structure d'une énumération

```
enum-key enum-name : enumerator-type { };
```

Enum Name
Optional name of the enumeration

```
enum-key enum-name : enumerator-type { };
```

Enumerator Type
Can be omitted and the compiler will try to deduce it!

```
enum-key enum-name : enumerator-type { };
```

Enumerator List
List of enumerator definitions

```
enum {Red, Green, Blue};                // Implicit initialization
       0     1      2

enum {Red = 1, Green = 2, Blue = 3};    // Explicit initialization

enum {Red = 1, Green, Blue};            // Explicit/Implicit initialization
                 2      3
```

Liste des énumérations

Type d'énumération

```
enum {Red, Green, Blue};                // Underlying type: int
       0     1     2
      000   001   010

enum {Red, Green, Blue = -32769};       // Underlying type: long
       0     1    -32769
      000   001   11111111111111110111111111111111

enum : uint8_t {Red, Green, Blue};      // Underlying type: unsigned 8 bit int

enum : long long {Red, Green, Blue};    // Underlying type: long long
```

Integral Type	Width in bits
int	at least 16
unsigned int	
long	at least 32
unsigned long	
long long	at least 64
unsigned long long	

Figure 1: énumération

Nom de l'énumération

Énumérations sans portée

- Utilisation des instructions `if` et `switch` avec des énumérations sans portée
- Utilisation de `cin` et `cout` avec des énumérations sans portée

Énumérations ciblées

Motivation

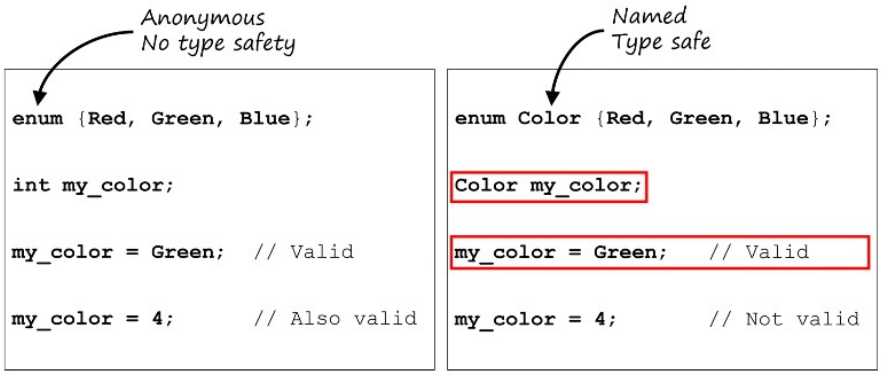

Figure 2: énumération

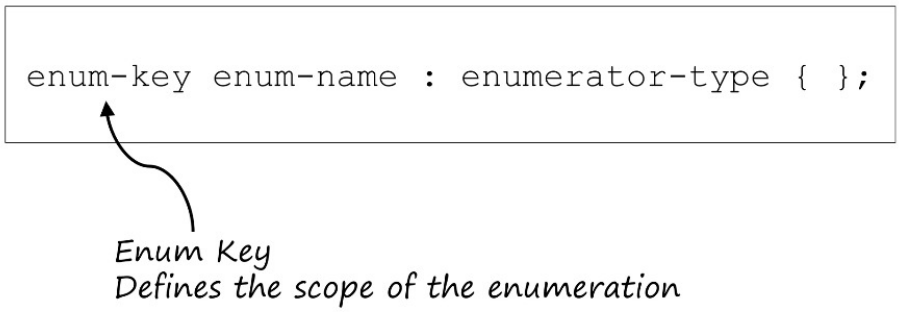

Figure 3: énumération

```
State state = get_state();

if (state == Nominal)                    // Accessed first
    initiate(Launch);
else if (state == Inclement_Weather)     // Accessed second
    initiate(Hold);
else if (state == Engine_Failure)        // Accessed last
    initiate(Abort);
```

Figure 4: énumération

```
State state = get_state();

switch (state) {
    case Engine_Failure:       // Equal access time
        initiate(Abort);
        break;
    case Inclement_Weather:    // Equal access time
        initiate(Hold);
        break;
    case Nominal:              // Equal access time
        initiate(Launch);
que ici.
}
```

Figure 5: énumération

```
enum State {Engine_Failure, Inclement_Weather, Nominal};

State state;
std::cin >> state;    // Not allowed using standard extraction operator
```

Figure 6: énumération

```
enum State {Engine_Failure, Inclement_Weather, Nominal};    // Underlying type: int
                 0                 1             2
std::underlying_type_t<State> user_input;                    // Type: int
std::cin >> user_input;                                      // userInput = 3
State state = State(user_input);                             // state = 3
```

Figure 7: énumération

```
enum State {Engine_Failure, Inclement_Weather, Nominal};    // Underlying type: int

std::istream& operator>>(std::istream& is, State& state) {
    std::underlying_type_t<State> user_input;                // Type: int
    is >> user_input;
    switch (user_input) {
        case Engine_Failure:       state = State(user_input); break;
        case Inclement_Weather:    state = State(user_input); break;
        case Nominal:              state = State(user_input); break;
        default:                   std::cout << "User input is not a valid state.";
    }
    return is;
}
```

Figure 8: énumération

```
enum State {Engine_Failure, Inclement_Weather, Nominal};

State state = Engine_Failure;

std::cout << state;    // Displays 0
```

Figure 9: énumération

```
enum State {Engine_Failure, Inclement_Weather, Nominal};

std::ostream& operator<<(std::ostream& os, State& state) {
    switch (state) {
        case Engine_Failure:       os << "Engine Failure"; break;
        case Inclement_Weather:    os << "Inclement Weather"; break;
        case Nominal:              os << "Nominal"; break;
        default:                   os << "Unknown";
    }
    return os;
}
```

Figure 10: énumération

```
State state = Engine_Failure;
std::cout << state;              // Displays "Engine Failure"
```

Figure 11: énumération

```
enum enum-name : enumerator-type { };
```

Enum
Defines an unscoped enumeration

Figure 12: énumération

```
enum class enum-name : enumerator-type { };
```

enum class or enum struct
Defines a scoped enumeration

Figure 13: énumération

```
enum Whale {Blue, Beluga, Gray};
enum Shark {Greatwhite, Hammerhead, Bull, Blue};

if (Beluga == Hammerhead)
    std::cout << "A beluga whale is equivalent to a hammerhead shark.";
```

Figure 14: énumération

- Utilisation des instructions `if` et `switch` avec des énumérations avec portée

```
enum class Whale {Blue, Beluga, Gray};

Whale whale = Whale::Beluga;

if (whale == Whale::Blue)
    std::cout << "Blue whale";
else if (whale == Whale::Beluga)
    std::cout << "Beluga whale";
else if (whale == Whale::Gray)
    std::cout << "Gray whale";
```

```
enum class Whale {Blue, Beluga, Gray};

Whale whale = Whale::Beluga;

switch (whale) {
    case Whale::Blue:
        std::cout << "Blue whale"; break;
    case Whale::Beluga:
        std::cout << "Beluga whale"; break;
    case Whale::Gray:
        std::cout << "Blue whale"; break;
}
```

Figure 15: énumération

- Utilisation des valeurs de l'énumérateur étendu

```
enum class Item {Milk = 350, Bread = 250, Apple = 132};    // Underlying type: int

int milk_code = Item::Milk;                    // Error! (Cannot convert Item to int)
int total = Item::Milk + Item::Bread;          // Error! (No matching '+' operator)
std::cout << Item::Milk;                       // Error! (No matching '<<' operator)
```

```
enum class Item {Milk = 350, Bread = 250, Apple = 132};    // Underlying type: int

int milk_code = int(Item::Milk);                           // milk_code = 350
or
int milk_code = static_cast<int>(Item::Milk);

int total = int(Item::Milk) + int(Item::Bread);            // total = 600
std::cout << underlying_type_t<Item>(Item::Milk);          // Displays 350
```